

Module 1: File Basics (The File Class) - Complete Study Notes

Welcome to **Module 1** of our Java File Handling course! This module focuses on the foundational `File` class from the `java.io` package. The `File` class represents an abstract pathname for files and directories—think of it as a handle to a file on your disk without actually opening or reading its contents. It's perfect for metadata operations like checking if a file exists, getting its size, or managing directories.

By the end of this module, you'll:

- Create and manipulate `File` objects.
- Perform basic filesystem checks and operations.
- Build simple methods for directory traversal.
- Understand best practices for error handling.

Key Concepts:

- `File` is **not** for reading/writing data (that's for streams in later modules).
- Paths can be relative (e.g., "data.txt") or absolute (e.g., "/home/user/data.txt").
- Always handle `IOException` for safety—filesystem ops can fail (e.g., permissions).

Prerequisites: Basic Java (variables, methods, exceptions). We're using Java 8+.

1. Core Concepts: What is a File Object?

A `File` object is like a pointer to a file or directory. It doesn't load the file into memory; it just knows its location. This makes it efficient for quick checks.

Import Statement (add at top of every class):

```
import java.io.File; // Core class for file metadata
import java.io.IOException; // For exception handling
```

2. Step-by-Step Code Examples with Comments

Let's build examples progressively. Each snippet is a complete, runnable method or class. Run them in your IDE (e.g., Eclipse) to see output.

Step 1: Creating a File Object

This is the simplest step—instancing a `File` points to a location.

Sample Code: `BasicFileCreator.java`

```
public class BasicFileCreator {
    public static void main(String[] args) {
        // Create a File object for a relative path (looks in current working directory)
        File file = new File("example.txt");

        // Alternative: Absolute path (full path from root)
        // File file = new File("/Users/yourname/Documents/example.txt"); // macOS/Linux
        // File file = new File("C:\\Users\\yourname\\Documents\\example.txt"); // Windows (double backslashes)

        // Print the path to verify
        System.out.println("File path: " + file.getAbsolutePath()); // Gets full absolute path
        System.out.println("File name: " + file.getName()); // Just the filename (e.g., "example.txt")
    }
}
```



Output Example:

```
File path: /path/to/your/project/example.txt
File name: example.txt
```

Why Comment? Paths differ by OS—always use `getAbsolutePath()` to debug.

Step 2: Checking File Properties

Use methods like `exists()`, `length()`, `canRead()` to inspect without opening the file.

Sample Code: `FileInspector.java` (Extends Step 1)

```

public class FileInspector {
    public static void main(String[] args) {
        File file = new File("example.txt"); // Assume this file may or may not exist

        // Check if the file or directory exists on the filesystem
        if (file.exists()) {
            System.out.println("File exists!");

            // Get size in bytes (0 for empty files/directories)
            long sizeInBytes = file.length();
            System.out.println("Size: " + sizeInBytes + " bytes (" + (sizeInBytes / 1024.0) + " KI

            // Check permissions: Can we read/write?
            System.out.println("Readable: " + file.canRead());
            System.out.println("Writable: " + file.canWrite());

            // Is it a file or directory?
            System.out.println("Is file: " + file.isFile()); // true for regular files
            System.out.println("Is directory: " + file.isDirectory()); // true for folders
        } else {
            System.out.println("File doesn't exist—create it first!");
        }
    }
}

```

Output Example (if file exists and is 10 bytes):

```

File exists!
Size: 10 bytes (0.009765625 KB)
Readable: true
Writable: true
Is file: true
Is directory: false

```

Pro Tip: `length()` returns 0 for directories—use `listFiles()` (later) for dir contents.

Step 3: Creating and Deleting Files/Directories

`createNewFile()` makes an empty file; `mkdirs()` creates parent dirs if needed. `delete()` removes immediately.

Sample Code: FileManager.java (Combines creation/deletion)

```
public class FileManager {  
    public static void main(String[] args) {  
        File file = new File("data/example.txt"); // Nested path—parent "data" dir may not exist  
  
        try {  
            // Create parent directories recursively (safe if they already exist)  
            File parentDir = file.getParentFile();  
            if (parentDir != null) {  
                parentDir.mkdirs(); // Creates "data/" if missing; no error if exists  
                System.out.println("Parent dir created: " + parentDir.exists());  
            }  
  
            // Create the empty file (returns true if successful, false if already exists)  
            boolean created = file.createNewFile();  
            System.out.println("File created: " + created + " (Path: " + file.getAbsolutePath() +  
  
            // Now delete it (returns true if successful)  
            boolean deleted = file.delete();  
            System.out.println("File deleted: " + deleted);  
  
        } catch (IOException e) {  
            // Handle errors like permission denied or disk full  
            System.err.println("IO Error: " + e.getMessage());  
            e.printStackTrace(); // Prints stack trace for debugging  
        }  
    }  
}
```

**Output Example:**

```
Parent dir created: true  
File created: true (Path: /path/to/project/data/example.txt)  
File deleted: true
```

Error Handling Note: `createNewFile()` and `mkdirs()` throw `IOException` —always use try-catch!

3. Practice: Building Reusable Methods

Now, let's create utility methods. This "classwork" helps you think modularly.

Classwork 1: Directory Lister

Write a method to list all files in a directory. Use `listFiles()` which returns a `File[]` array.

Complete Class: DirectoryLister.java

```

import java.io.File;

public class DirectoryLister {
    /**
     * Lists all files and subdirectories in the given path.
     * @param path The directory path as a string
     */
    public static void listDirectoryContents(String path) {
        File directory = new File(path); // Create File object for the directory

        // Check if it's a valid directory
        if (!directory.exists() || !directory.isDirectory()) {
            System.out.println("Invalid directory: " + path);
            return; // Early exit if not a dir
        }

        // Get array of File objects for contents
        File[] contents = directory.listFiles(); // null if error; includes files and subdirs

        if (contents != null) {
            System.out.println("Contents of " + path + ":");
            for (File item : contents) { // Loop through each item
                String type = item.isDirectory() ? "DIR" : "FILE"; // Label type
                System.out.println(type + ": " + item.getName() + " (" + item.length() + " bytes)");
            }
        } else {
            System.out.println("Could not list contents—check permissions.");
        }
    }

    public static void main(String[] args) {
        listDirectoryContents("."); // "." means current directory
        // listDirectoryContents("/Users/yourname/Documents"); // Example absolute path
    }
}

```

Sample Output (in a project dir):

Contents of .:

FILE: DirectoryLister.java (456 bytes)

DIR: data (0 bytes)

FILE: example.txt (0 bytes)

Customization: Modify to sort by name:

```
Arrays.sort(contents, (a, b) -> a.getName().compareTo(b.getName())); (import java.util.Arrays ).
```

Depth Adder: Filtering Files (e.g., Only .txt)

Extend `listFiles()` with a `FileFilter` for selective listing.

Enhanced Class: TxtFileLister.java

```
import java.io.File;
import java.io.FileFilter;

public class TxtFileLister {
    /**
     * Lists only .txt files in the directory using a FileFilter.
     * @param path The directory path
     */
    public static void listTxtFiles(String path) {
        File directory = new File(path);

        if (!directory.exists() || !directory.isDirectory()) {
            System.out.println("Invalid directory: " + path);
            return;
        }

        // Anonymous inner class for FileFilter: accepts only files ending in .txt
        FileFilter txtFilter = new FileFilter() {
            @Override
            public boolean accept(File file) {
                // Check if it's a file (not dir) and ends with .txt (case-insensitive)
                return file.isFile() && file.getName().toLowerCase().endsWith(".txt");
            }
        };

        // Pass filter to listFiles()—only matching Files returned
        File[] txtFiles = directory.listFiles(txtFilter);

        if (txtFiles != null && txtFiles.length > 0) {
            System.out.println("TXT files in " + path + ":");
            for (File txtFile : txtFiles) {
                System.out.println("- " + txtFile.getName() + " (" + txtFile.length() + " bytes)");
            }
        } else {
            System.out.println("No .txt files found.");
        }
    }

    public static void main(String[] args) {
        listTxtFiles("."); // List in current dir
    }
}
```

```
}  
}
```

Why Filter? Efficient for large dirs—avoids loading everything.

4. Tasks and Exercises

Apply what you've learned! Complete these in order. Submit as .java files or run outputs.

Task 1: Basic Inspector (Beginner - 10 mins)

- Create a class `MyFileChecker` .
- Method: `checkFile(String filename)` that prints: exists?, size (in KB), readable?, absolute path.
- Test with a real file (e.g., create "test.txt" first).
- **Expected:** Handle non-existent file gracefully (print "Not found").

Task 2: Directory Creator (Intermediate - 15 mins)

- Class: `DirBuilder` .
- Method: `buildNestedDir(String fullPath)` that uses `mkdirs()` on parents and creates an empty file at the end.
- Test: `buildNestedDir("logs/2023/oct.txt")` —verify with `ls` or File Explorer.
- **Challenge:** Add param to make it writable-only (use `setWritable(false, false)`).

Task 3: Recursive Lister (Advanced - 20 mins)

- Extend `DirectoryLister` to recurse into subdirs (hint: call `listDirectoryContents(item.getAbsolutePath())` if `isDirectory()`).
- Limit depth to 1 (avoid infinite loops).
- **Bonus:** Count total files vs. dirs.

Task 4: Error Simulator (Debugging - 10 mins)

- Modify `FileManager` to force an error (e.g., create in read-only dir).
- Catch and log: "Error type: [e.getClass().getSimpleName()], Message: [e.getMessage()]" .
- **Why?** Teaches robust code.

Grading Rubric (Self-Assess):

- Code compiles/runs: 40%
- Comments explain each line/block: 30%
- Handles edge cases (e.g., null paths): 20%
- Clean output: 10%

5. Common Pitfalls and Best Practices

Pitfall	Fix
Forgetting try-catch on <code>createNewFile()</code>	Always wrap in <code>try { } catch (IOException e) { }</code>
Relative paths not working	Use <code>System.getProperty("user.dir")</code> to print current dir
Windows backslashes	Use forward slashes <code>/</code> or double <code>\\</code> —Java normalizes
Listing null dirs	Check <code>if (contents != null)</code> before looping

Pro Tip: For production, use `java.nio.file.Path` (Module 4) over `File` —it's more modern.

Resources:

- Oracle Docs: [java.io.File](#)
- Practice Online: LeetCode or HackerRank file I/O problems.

Next Up: Module 2 (Text Handling). Complete tasks before proceeding—questions? Reply with your code!